VIETNAM–KOREA UNIVERSITY OF INFORMATION AND COMMUNICATION TECHNOLOGY (VKU)
Faculty of Computer Science

MINI-PROJECT SHORT TECHNICAL REPORT

NGÔN SẮC VIỆT

Offline-first translation core for Vietnamese ethnic minority languages

Course	Cross-Platform Mobile App Development (VKU)
Mini-Project Title	*[Mini-Project 1 / 2 / 3]* – Offline-first translation core: local persistence, background sync and AI translation API
Team Name	Ngôn Sắc Việt
Submission Date	*[DD/MM/YYYY]*

1. GENERAL INFORMATION & DELIVERABLE LINKS

Team members:

No.	Full Name	Student ID	Main role	Part in the Flutter app	Contribution
1	Bùi Thị Lệ Thắm	23IT253	Team leader, backend (FastAPI, Supabase PostgreSQL); coordinates work and user flows; extensible architecture	Model and API layer: freezed models, dio ApiClient, token interceptor, error mapping, native integrations (ML Kit OCR, secure storage, file picker)	25%
2	Nguyễn Minh Duy	23IT038	AI model with Nguyên: research on the five languages, text translation model first, then speech models	Base code: clean architecture, Riverpod wiring, build-time configuration, loading the bundled catalogue into the database	25%
3	Lương Thế Nguyên	23IT183	AI model with Duy: datasets and training pipeline, speech recognition and OCR	Local database and sync: SQLite store, LocalStore, offline queue, catalogue sources, SyncUseCase and its tests	25%
4	Trần Đình Nhứt	23IT203	Frontend lead: localization, light and dark mode, go_router, model generation, current Flutter libraries	UI from the Figma design, connecting screens to data, final integration of all parts	25%

The Flutter app was built by all four members in sequence. Duy set up the base code and the database start-up; Thắm completed the models, the API client and the native parts; Nguyên added the local database and synchronisation; Nhứt built the screens from the design, connected them to the data and integrated everything. Each module was also tested by a member who did not write it.

Live demo URL: *[https://your-demo.pages.dev or APK download link]*

GitHub repository: *[https://github.com/username/your-repo-name]*

Video demo (optional): *[https://youtu.be/xxx]*

2. FEATURE IMPLEMENTATION CHECKLIST

"Complete" means implemented and covered by at least one automated test. "Partial" means the main logic works but part of the scope listed in the proposal is not finished yet.

No.	Required Feature	Status	Implementation Details & Acceptance Level
1	Responsive mobile viewport, dark mode, localization	Complete	Material 3 light and dark themes plus system mode; Vietnamese and English through ARB files (gen-l10n). The user's choice is saved with shared_preferences and applied at start-up. Layouts use scrollable lists and a bottom navigation bar, so they adapt to phone widths.
2	Local offline persistence	Complete	SQLite table documents(collection, id, payload) plus a meta table. LocalStore gives typed access to languages, peoples, heritage, lessons and contributions. On first launch the app loads a bundled catalogue (assets/catalog.json), so it works offline right after installation.
3	Automatic background sync	Partial	SyncUseCase pulls the catalogue and pushes pending contributions. It runs when the app shell opens, after each new contribution and from the Sync button. Not yet done: running it automatically when the connection comes back.
4	Text translation (ethnic languages, Vietnamese, English)	Complete	Direct route or route through Vietnamese; reviewed glossary first, optional mT5 or NLLB model. The response includes route, engine, steps and warnings. A direction with no reviewed data returns HTTP 422 model_not_ready instead of an invented sentence.
5	Photo translation (OCR)	Partial	On Android and iOS the Translate screen reads text from a picked image with Google ML Kit (Latin script) and puts it into the input box for editing. Web and Windows show a "not supported" message. The server endpoint /v1/translate/image exists; the server OCR model is not loaded yet.
6	Voice translation (ASR)	Partial	Endpoint /v1/translate/speech accepts audio up to 20 MB and chains recognition, translation and logging. The Whisper model is being fine-tuned; there is no recording screen in the app yet.
7	Authentication and roles	Complete	Register, login, refresh, logout and profile. Access token is a JWT (HS256); the refresh token is stored only as a SHA-256 hash and replaced on every refresh. Passwords use Argon2. Roles: member and reviewer.
8	Community contribution and review	Complete	A contribution is saved locally as pending with a UUID client ID, then uploaded. Reviewers approve or reject with a note; approved pairs are included in sync and exported as CSV for training. The review screen in the app is not built yet; reviewers use the API.
9	Heritage, lessons and quiz	Complete	Heritage list and detail per people; lesson list and quiz screen. The quiz is scored on the server and the answer key is never stored on the phone.
10	Database schema and security	Complete	Supabase migration 0001_init.sql: 11 tables with foreign keys, Row-Level Security on 8 tables. The public key can only read the catalogue and verified phrases.
11	Automated tests and static analysis	Complete	21 tests pass: 7 backend, 8 AI service, 6 Flutter. flutter analyze reports no issues.

3. TECHNICAL ARCHITECTURE & PROJECT STRUCTURE

3.1. Directory structure

The repository contains three services and one shared catalogue. Each service has its own dependency file, Dockerfile and tests.

```
ai4life/
|-- fe/                          Flutter app (Dart 3.8)
|   |-- lib/
|   |   |-- data/                LocalStore, SQLite box, Riverpod wiring
|   |   |   `-- services/        ApiClient, catalog sources, OCR, tokens, prefs
|   |   |-- domain/              freezed models, SyncUseCase, RecognizeTextUseCase
|   |   |-- l10n/                app_en.arb, app_vi.arb (generated localizations)
|   |   `-- ui/
|   |       |-- core/            app (GoRouter), app_shell (tabs), app_theme
|   |       `-- features/        translate, heritage, learn, contribute,
|   |                            settings, auth (views and view models)
|   |-- assets/catalog.json      bundled catalogue for the first offline launch
|   `-- test/
|-- be/                          API gateway: FastAPI, SQLAlchemy 2 async
|   |-- app/core/                config, db, deps, security (JWT, Argon2)
|   |-- app/routers/             auth, catalog, translate, learn, community
|   |-- app/integrations/        ml_client (HTTP client to the AI service)
|   |-- supabase/migrations/     0001_init.sql (11 tables, RLS)
|   `-- tests/
|-- ai-model/                    AI service: FastAPI
|   |-- app/                     detect, translate, OCR and ASR adapters
|   |-- data/reviewed_pairs.csv
|   |-- training/                finetune_mt5.py, validate_pairs.py
|   `-- tests/
`-- shared/languages.json        7 languages, 5 peoples, heritage texts
```

Ownership inside fe/lib: the folder structure, providers, main.dart and open_box were set up by Duy; domain/models, services/api_client, token_store and the OCR engines by Thắm; sqlite_document_box, local_store, catalog_sources and sync_use_case by Nguyên; ui/, l10n/ and app_theme by Nhứt.

3.2. Layers in the Flutter app

The app follows a simple MVVM split. Views only build widgets. View models are Riverpod Notifiers that receive user actions (for example TranslateViewModel and SettingsViewModel). The data layer contains LocalStore, ApiClient and the OCR engine. Use cases exist only where there is a real rule to enforce: SyncUseCase decides what is pulled and pushed, and RecognizeTextUseCase trims OCR output and rejects empty pages.

The domain layer depends on small interfaces (CatalogSource, ContributionSink, OcrEngine, DocumentBox) instead of concrete classes. In main.dart the real implementations are injected through ProviderScope overrides; in tests the same use cases run with in-memory fakes. Platform differences are handled with conditional imports: ML Kit is used on Android and iOS, and a stub that reports "not supported" is used on web and desktop.

The catalogue can come from two sources. By default the app calls GET /v1/sync/pull on the API. If SUPABASE_URL and SUPABASE_ANON_KEY are given at build time, it reads the public tables directly from Supabase, using the view quiz_choices_public so the is_correct column is never downloaded.

3.3. State management and data flow

1. Start-up. main.dart opens SQLite. If the meta flag has_catalog is not set, the bundled assets/catalog.json is written into the local store. Theme and language are read from shared_preferences. The UI then renders from local data without waiting for the network.
2. First sync. After the first frame, AppShell calls SyncUseCase.run(). It downloads the snapshot, replaces each local collection, and pushes any pending contributions. The FutureProviders for languages, peoples, lessons and contributions are then invalidated so the screens reload from SQLite.
3. Contribution. The Contribute screen writes the contribution locally with status pending and a new UUID, then triggers a sync. If the upload succeeds the server copy, with its server ID, replaces the local one. If it fails, the item stays in the queue and the screen shows "saved offline".
4. Translation. TranslateViewModel calls ApiClient.translateText and stores loading, result or error in its state. The gateway checks the language codes, calls the AI service, writes a row to translation_logs and returns the result.

3.4. Exception handling

All API errors use the same body, {code, message}, with a Vietnamese message. On the app side ApiClient.guard turns a DioException into an ApiException holding that message and the HTTP status, so screens show a readable sentence instead of a stack trace.

Where	Situation	Behaviour
AI service	Unknown language code or empty text	HTTP 400 unknown_language or empty_text.
AI service	No reviewed glossary or model for the direction	HTTP 422 model_not_ready with the source and target codes.
AI service	OCR or ASR model not loaded	HTTP 200 with empty text and a warning, so the app can explain the situation.
API gateway	AI service fails or times out	400 and 422 are passed through; other failures become 503.
API gateway	File too large	HTTP 413 file_too_large (image over 8 MB, audio over 20 MB).
API gateway	The same contribution uploaded twice	Same client ID from the same user returns the existing row; from another user returns HTTP 409.
API gateway	Member calls a reviewer endpoint	HTTP 403 forbidden.
Flutter app	No network during sync	SyncUseCase catches the error and returns SyncReport(online: false). Local data and the queue are untouched; the shell shows that work was saved offline.
Flutter app	OCR on web or desktop, or no text found	UnsupportedError and OcrEmptyException are caught and shown as a short message.

4. EMPIRICAL EVIDENCE & SCREENSHOTS

4.1. Automated tests

Run on Windows 11 with Python 3.11 and Flutter (Dart 3.8), each Python service in its own virtual environment:

```
be/        python -m pytest -q        7 passed
ai-model/  python -m pytest -q        8 passed
fe/        flutter test               6 passed  (All tests passed!)
fe/        flutter analyze            No issues found!
```

Test file	Tests	What is checked
be/tests/test_api.py	6	Health check and five seeded peoples; register and refresh-token rotation; text translation and image fallback; lesson does not expose the answer key; contribution, review and CSV export; sync pull contains the catalogue.
be/tests/test_scoring.py	1	Quiz scoring counts only the correct choice.
ai-model/tests/test_core.py	8	Five languages in the catalogue; direct glossary translation; route through Vietnamese; ethnic direction not invented; route planning; detection of Cham script, Vietnamese and unknown Latin text; unverified pairs rejected; training dry run.
fe/test/sync_test.dart	5	Bundled catalogue mapping; Supabase rows do not keep the answer key; contribution kept locally when offline; pending contribution uploaded and given a server ID; OCR text trimmed and empty page rejected.
fe/test/widget_test.dart	1	App shell opens on the Translate tab.

4.2. API responses

Vietnamese text with automatic detection, translated to English through the reviewed glossary:

```
POST /v1/translate  {"text": "Xin chào", "source_lang": "auto", "target_lang": "en"}
200 OK
{"text": "Hello", "source_lang": "vi", "target_lang": "en", "detected_lang": "vi",
 "route": "direct", "engine": "glossary", "warnings": [],
 "steps": [{"source": "vi", "target": "en", "engine": "glossary", "text": "Hello"}]}
```

Tày to English before the Tày model has been trained. The service refuses rather than guessing:

```
POST /v1/translate  {"text": "...", "source_lang": "tyz", "target_lang": "en"}
422 Unprocessable Entity
{"detail": {"code": "model_not_ready",
 "message": "Chưa có mô hình hoặc cặp câu đã rà soát cho hướng dịch này.",
 "source_lang": "tyz", "target_lang": "en"}}
```

4.3. UI design (Figma)

The screens are implemented from the team's Figma design (https://www.figma.com/design/aFYvfKRuZdLQAjwq8Yfu3j/VietLangAI--Community-). The four frames below correspond to the four tabs of the app's bottom navigation bar and to the routes /translate, /heritage, /learn and /contribute.

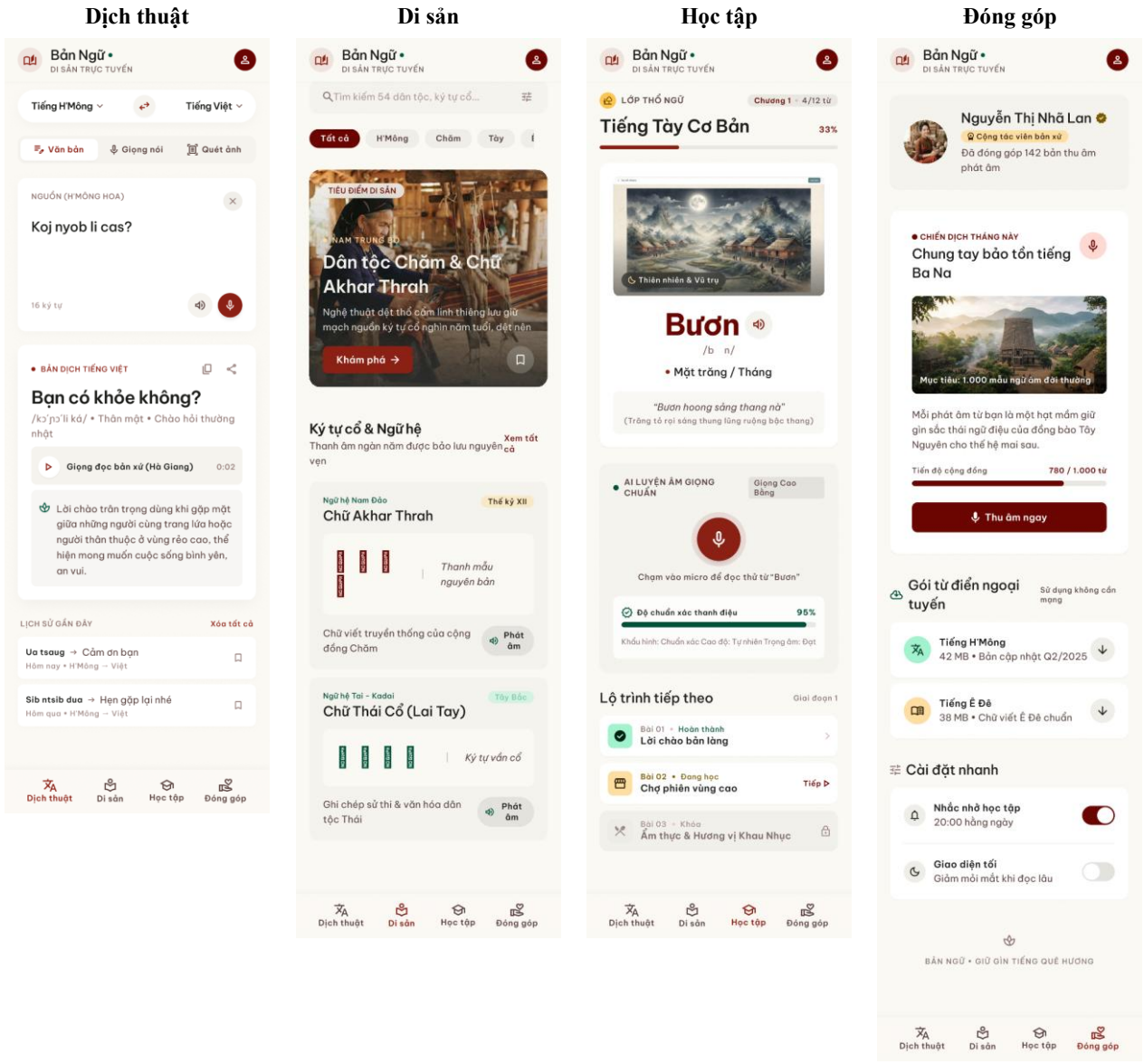


Figure 1. Figma design of the four main screens: translation with text, voice and photo input; heritage and old scripts; basic lessons with AI pronunciation practice; community contribution and offline dictionary packs.

4.4. Screenshots of the running app

[Insert screenshot]
Android emulator or phone

Figure 2. Translate tab: language selectors, text read from a photo, result with warnings (light theme).

[Insert screenshot]
Two screenshots side by side, airplane mode on and off

Figure 3. Contribute tab: a contribution saved while offline and shown as pending; the same item after sync.

[Insert screenshot]
Dark theme

Figure 4. Heritage detail for one people (region, language family, origin of the language).

[Insert screenshot]
English interface

Figure 5. Quiz screen after submitting, with score; Settings screen with theme and language options.

5. TECHNICAL CHALLENGES & RESOLUTIONS

5.1. Low-resource languages and invented translations

Public parallel data for Chăm and Jrai is very limited, and the Latin-script ethnic languages share most letters with Vietnamese. A detector based only on characters cannot reliably tell a Tày or Ê Đê sentence from Vietnamese, and a multilingual model that has never seen these languages tends to produce fluent Vietnamese that does not match the input. For H'Mông, most text found online uses the overseas RPA spelling, which is not what users in Vietnam write.

We changed the design so that the system only answers when it has reviewed support:

- The route planner builds a direct route or a route through Vietnamese, and raises RouteNotReady when any step is not covered. The API turns this into HTTP 422 model_not_ready, and the app shows it as a warning.

- The reviewed glossary is consulted before the neural model. Rows not marked as verified are skipped when the glossary is loaded, and validate_pairs.py applies the same rule before fine-tuning.
- The detector recognises Cham only from its Unicode block and does not guess between Latin-script ethnic languages; the user chooses the source language until a trained language-ID model is ready.
- Training data for H'Mông is restricted to the Vietnamese orthography.

As a result the app sometimes says "not supported yet", but every translation it does show can be traced to reviewed data or a named model.

5.2. Duplicate data during offline sync

A contribution written offline can be uploaded more than once. The connection may drop after the server has saved it but before the phone receives the reply, or the user may press Sync twice. Without protection the same pair would appear several times in the review queue and later in the training set, which would bias the model.

Each contribution gets a UUID on the phone, stored as client_id with a UNIQUE constraint in PostgreSQL. The push endpoint is idempotent: a client_id already saved by the same user returns the existing row, and one belonging to another user returns HTTP 409. The phone only treats an item as uploaded when the server returns it with a server ID, and it then overwrites the local copy with the server version. Retrying is therefore always safe. Two Flutter tests cover the offline case and the successful upload.

5.3. Reproducible environment

When one member set up the backend in a new virtual environment, the server failed to start with "the SQLAlchemy asyncio module requires the greenlet library". After this was fixed, two tests still failed because the current version of email-validator rejects the reserved .local domain that we used for the seeded demo account, which would also have broken the demo login. We changed the dependency to sqlalchemy[asyncio], moved the demo and test accounts to a valid example domain, and now run the full test suite on a clean environment before merging.

Next steps

- Run sync automatically when the connection returns, and add the reviewer screen to the app.
- Fine-tune mT5-small on reviewed Tày and Ê Đê pairs and compare BLEU and chrF with the glossary baseline.
- Fine-tune Whisper on H'Mông VOV audio and add the recording screen for voice translation.